# Skills, Agents & Automation

## Building Reusable AI Workflows

Week 8 — Phase 2: Engineering

*"From one-shot questions to automated pipelines."*

Instructor: Goker Ezberci

Vibe Coding 101: From Idea to Shipped Product

github.com/goker/vibe-coding-101-for-software-engineers

# Today's Agenda

01 **From One-Shot to Workflow**
Question-answer vs. automated pipelines

02 **Claude Code Skills**
Custom commands with SKILL.md structure

03 **MCP Protocol**
Tools, Resources, Prompts — standards for AI integration

04 **Agent Design Patterns**
Reflection, Tool Use, Planning, Multi-Agent (Andrew Ng)

05 **Building Your DevOps Agent**
4-step pipeline: git → tests → commit → PR

06 **Lab Exercise**
Create a skill, build a tool, chain into workflow

# From One-Shot to Workflow

*Shift from isolated queries to automated pipelines*

## One-Shot (Basic)

```
User: "Write me a function
          to parse JSON"

AI: [Returns code]

User: [Copy-paste]
      [Use once]
      [Move on]
```

## Workflow (Advanced)

```
User: "Deploy app"

Pipeline:
1. git status check
2. Run tests
3. Lint code
4. Generate commit
5. Create PR
```

### AI as a Component, Not Just a Chatbot

One-shots are instant. Workflows are repeatable, scalable, and save hours every month.

# Claude Code Skills

*Custom commands with SKILL.md frontmatter + markdown instructions*

## What's a Skill?

- Custom command: /commit-message
- Describes what & when to use
- Shows input/output examples
- Calls tools, scripts, APIs

```
# Commit Message Skill

## Command
/commit-message

## Description
Reads git diff, generates
conventional commit message

## When to Use
After staging with git add
```

**Reference:** code.claude.com/docs/en/skills

Skills auto-invoke when relevant. Consistent formatting without context-switching.

# MCP: Model Context Protocol

*Open standard for connecting AI systems to tools, data, and templates*

### Tools

Functions/APIs

```
run_tests(path)
```

### Resources

Data/State

```
git_status
```

### Prompts

Templates

```
pr_template(commits)
```

## Simple MCP Server (Node.js)

```
const server = new stdio.Server({
  tools: [
    { name: "run_tests",
      description: "Execute test suite",
      handler: async (input) => {
        const result = execSync(`npm test`);
        return { status: "success", output: result };
      }
    }
  ]
});
```

Reference: modelcontextprotocol.io

# Agent Design Patterns (Andrew Ng)

*Four core patterns for building AI systems — 2x2 grid*

### Reflection

AI reviews its own output

→ *Commit message validator*

### Tool Use

AI calls external APIs

→ *Test runner + linter*

### Planning

AI breaks tasks into steps

→ *Full CI/CD workflow*

### Multi-Agent

Multiple AIs coordinate

→ *Code review + test team*

# Building Your Personal DevOps Agent

*4-step automated pipeline: git → tests → commit → PR*

### 1 Git Status Reader

Read staged changes

`git diff --cached`

### 2 Test Runner

Verify tests pass

`npm test`

### 3 Commit Generator

Create commit message

`Claude API`

### 4 PR Writer

Generate PR description

`Claude API`

**Complete Workflow:**

User stages changes → Agent reads diff → Checks tests → AI generates message → AI creates PR → Done

```
$ devops-agent --workflow commit
▫ Tests passed  •  ▫ Suggested: feat(auth): add JWT refresh token  •  ▫ PR created
```

# References & Further Reading

## MCP & Integration

Anthropic MCP

Code Execution with MCP

MCP Servers Repo

## Claude Code & Skills

Claude Code Skills Docs

Skills Best Practices

Claude API Reference

## Agent Design

Andrew Ng - Agentic AI

LangGraph Documentation

## Orchestration

CrewAI Framework

LangChain Agents

# Lab Exercise

## Part A

Create a Skill

- ☐ SKILL.md file
- ☐ Command: /your-skill
- ☐ Description section
- ☐ When to Use section
- ☐ Input/Output examples

## Part B

Build MCP Tool

- ☐ Choose a tool
- ☐ Input schema
- ☐ Handler function
- ☐ Error handling
- ☐ Test it works

## Part C

Chain Workflow

- ☐ Link 2+ tools
- ☐ Pass output to input
- ☐ AI decision point
- ☐ Final result output
- ☐ Document flow

## Submission

Push to repo: SKILL.md • mcp-tool.js • workflow.py/js with full documentation

# Key Takeaways

1. Workflows > One-shots — Reusable systems beat one-time AI queries
2. Skills + MCP — Combine Claude Code Skills with MCP tools for automation
3. Design Patterns Matter — Reflection, Tool Use, Planning, Multi-Agent are your toolkit
4. Build Real DevOps Agents — 4-step pipeline saves hours every month
5. Composition is King — Chain simple tools into complex, reliable workflows

Course Materials: github.com/goker/vibe-coding-101-for-software-engineers